



UNIVERSIDAD CATÓLICA DEL NORTE
FACULTAD DE INGENIERÍA Y CIENCIAS
GEOLÓGICAS
DEPARTAMENTO DE INGENIERÍA DE SISTEMAS Y
COMPUTACIÓN

Extracción Iterativa de Conos Invertidos Positivos
en un Modelo de Bloques 3D

Integrantes:

Camilo Jofré Tapia

Jhoan Montero Huarachi

Sebastián Hernández Lara

Profesor:

Aldo Quelopana Retamal

Diseño y Análisis de Algoritmos

17/07/2026

Ingeniería Civil en Computación e Informática

Introducción

La planificación minera a cielo abierto constituye uno de los problemas de optimización combinatoria más complejos y relevantes de la industria extractiva. En este contexto, la determinación del orden óptimo de extracción de bloques no solo debe considerar su valor económico individual, sino también las restricciones geométricas y de estabilidad que condicionan qué bloques pueden removerse en cada momento del proceso productivo. Estas restricciones generan relaciones de precedencia entre bloques, cuya correcta modelación y explotación algorítmica es fundamental para maximizar el valor del yacimiento.

El presente trabajo aborda, desde la perspectiva del diseño y análisis de algoritmos, el problema de la extracción iterativa de conos invertidos positivos sobre un modelo de bloques tridimensional. Un cono invertido representa el conjunto mínimo de bloques que deben removerse para que un bloque objetivo pueda ser extraído de manera factible, respetando estrictamente las relaciones de precedencia entre bloques. El interés particular de este tema radica en que la extracción de conos positivos, aquellos cuya suma de valores económicos es estrictamente mayor que cero, debe realizarse de forma iterativa, actualizando el estado del yacimiento en cada paso, y deteniéndose cuando ya no existan conos positivos disponibles.

Este problema está directamente relacionado con el problema de la mina a cielo abierto, ampliamente estudiado en la investigación de operaciones y optimización combinatoria. Sin embargo, el enfoque adoptado en este trabajo no apunta a resolver el problema completo de planificación minera, sino a estudiar el comportamiento algorítmico de estrategias iterativas de extracción de conos, analizando su complejidad teórica y evaluando empíricamente su desempeño sobre instancias generadas a partir de modelos de bloques reales o sintéticos.

Esta entrega continúa el trabajo desarrollado en la Entrega 1, en la que se formuló el problema de extracción iterativa de conos invertidos positivos, se realizó la revisión bibliográfica y se propuso una línea base basada en el algoritmo de cono flotante (floating cone) implementado por fuerza bruta, con complejidad $O(n^3)$ en el peor caso.

El objetivo de esta segunda entrega es proponer y justificar una mejora algorítmica concreta respecto de esa línea base, entregar su pseudocódigo, realizar el análisis teórico correspondiente, implementarla de manera preliminar en C++ y compararla experimentalmente contra la línea base sobre instancias sintéticas reproducibles.

2. Propuesta algorítmica principal

2.1 Limitaciones identificadas en la línea base

La línea base (Algoritmo del Cono Flotante por fuerza bruta) utilizada para la entrega anterior funciona correctamente, sin embargo esta es ineficiente por tres razones principales:

- En cada iteración recorre todos los bloques no extraídos para buscar un candidato con cono positivo, aunque la mayoría de los conos no cambiaron de valor respecto de la iteración anterior.
- Reconstruye desde cero el cono invertido completo para cada candidato revisado, sin reutilizar el trabajo de iteraciones previas.
- Su política de selección, basada en el primer cono positivo encontrado, no considera la calidad relativa de los distintos conos disponibles. Esto puede conducir a un valor total extraído menor que el alcanzable con otra política de selección.

2.2 Algoritmo propuesto: Floating Cone Incremental

Considerando los tres puntos anteriores, se buscó utilizar un algoritmo el cual evite recomputaciones innecesarias, actualice información parcialmente y utilice mejores políticas de selección:

- Construcción de precedencias con acceso $O(1)$ por vecino: se usa una tabla hash que mapea la coordenada (x, y, z) al índice del bloque, evitando búsquedas lineales al construir la lista de los 9 predecesores de cada bloque.
- Caché incremental de conos con índice inverso: el valor y el conjunto de bloques del cono de cada candidato se calculan una sola vez y se guardan en memoria. Además se registra qué candidatos utilizan cada bloque. Cuando se extrae un cono, solo se invalidan y recalculan los candidatos cuyo cono compartía al menos un bloque con el cono extraído.
- Selección por máximo valor con cola de prioridad y eliminación perezosa: en vez de recorrer el modelo buscando el primer cono positivo, se mantiene un max-heap de candidatos con cono positivo, ordenado por valor cacheado. Cada entrada del heap lleva asociada una versión, si el candidato fue recalculado después de ser insertado en el heap, la entrada antigua se descarta al extraerla.

La propuesta prioriza reducir el tiempo de ejecución evitando recomputación redundante y cambia la política de selección de primer cono positivo a cono de mayor valor disponible. En los experimentos preliminares, esta modificación también mejoró el valor total extraído en una de las instancias

3. Pseudocódigo

A continuación se presenta el pseudocódigo del algoritmo propuesto. este reutiliza `construir_precedencias` y `calcular_cono` los cuales ya fueron definidos en la Entrega 1.

```
DEF Floating_Cone_Incremental(B):
    prec ← construir_precedencias(B)      // hash map O(1) por vecino
    R ← ∅, E ← ∅, Vtotal ← 0
    valorCache ← mapa vacío
    conoCache ← mapa vacío // candidato -> conjunto de bloques
    version ← mapa vacío (inicializado en 0)
    pertenece ← mapa vacío // bloque -> lista de candidatos que lo contienen
    heap ← cola de prioridad vacía (max-heap por valor)

    DEF recomputar(cand):
        version[cand] ← version[cand] + 1
        para cada bloque b en conoCache[cand]:
            pertenece[b].remove(cand)
        conoCache[cand] ← ∅
        si cand extraído o cand inextraíble: return
        cono ← calcular_cono(cand, R, prec)
        si cono es inextraíble: return
        valorCache[cand] ← Σ v(b) para b en cono
        conoCache[cand] ← cono
        para cada bloque b en cono: pertenece[b].add(cand)
        si valorCache[cand] > 0:
            heap.push((valorCache[cand], cand, version[cand]))

    // inicialización: un único cálculo de cono por candidato
    para cada b en B: recomputar(b)

    MIENTRAS heap no vacío:
        (valor, cand, v) ← heap.pop_max()
        SI cand extraído O v ≠ version[cand]: continue // entrada obsoleta

        extraidos ← conoCache[cand]
        R ← R ∪ extraidos ; E ← E ∪ extraidos
        Vtotal ← Vtotal + valor

        afectados ← ∅
        para cada b en extraidos:
            afectados ← afectados ∪ pertenece[b]
            pertenece[b] ← ∅
        afectados.remove(cand)

        para cada c en afectados: recomputar(c)

    RETORNAR E, Vtotal
```

4. Análisis teórico

4.1 Complejidad temporal

Sea $n = |B|$ el número total de bloques.

- Construcción de precedencias: $O(n)$, con hasta 9 accesos hash $O(1)$ amortizado por bloque.

- Inicialización: se calcula el cono de cada uno de los n candidatos exactamente una vez mediante DFS/BFS. En el peor caso, cuando los conos cubren una parte importante del modelo, este costo puede ser $O(n^2)$, pero se paga una sola vez y no se repite en cada iteración.
- Bucle principal: en cada extracción exitosa se recomputan únicamente los candidatos afectados, es decir, aquellos cuyo cono cacheado compartía bloques con el cono recién extraído. El índice inverso permite identificar estos candidatos en tiempo proporcional al tamaño del cono extraído.
- Cola de prioridad: las operaciones de heap cuestan $O(\log k)$, donde k es el número de entradas vivas. El borrado perezoso evita eliminar explícitamente entradas obsoletas.

En el peor caso teórico, por ejemplo si cada extracción afecta a todos los candidatos restantes, la cota superior puede mantenerse entre $O(n^2)$ y $O(n^3)$, igual que la línea base. La diferencia principal es que el trabajo del algoritmo mejorado es proporcional al número de candidatos afectados en cada iteración, y no al total de bloques del modelo. Cuando los conos son razonablemente locales, el número de candidatos afectados es sustancialmente menor que n .

4.2 Complejidad espacial

Además de los $O(n)$ bloques del modelo, el algoritmo mantiene en memoria los conos calculados para cada candidato y un índice inverso que relaciona cada bloque con los candidatos cuyos conos lo contienen. En el peor caso, si existe un alto nivel de solapamiento entre conos, esta información puede alcanzar un tamaño $O(n^2)$. Por lo tanto, el algoritmo utiliza más memoria que la línea base, pero a cambio reduce cálculos repetidos y mejora el tiempo de ejecución.

4.3 Casos límite y supuestos

- No se permiten bloques ficticios en los bordes del modelo. Un bloque no superficial al que le falta cualquiera de sus 9 predecesores queda marcado permanentemente como no extraíble y se excluye de recálculos futuros.
- Los conos de valor exactamente cero se detectan y registran mediante el contador `conosCero`, pero no se insertan en el heap ni se extraen.
- Si dos candidatos tienen exactamente el mismo valor cacheado, el orden de extracción queda determinado por el orden interno del heap. Esto no afecta la validez de la solución, aunque podría afectar marginalmente el valor final en instancias con muchos empates.

5. Argumento de validez

- Cada cono extraído se calcula con la misma función `calcular_cono` de la entrega anterior, por lo que utiliza exactamente la misma noción de cono invertido.
- La versión asociada a cada entrada del heap garantiza que solo se utiliza el estado actual del candidato y no un cálculo obsoleto.
- Solo se extraen conos con valor cacheado estrictamente positivo.
- Los bloques se marcan como extraídos dentro de un cono válido completo y se contabilizan una única vez.
- El heap solo contiene candidatos con valor positivo y cada extracción exitosa remueve al menos un bloque. Como B es finito, el proceso termina en a lo sumo n extracciones.

Esta propiedad se verificó adicionalmente mediante un validador independiente (`src/validate.cpp`), que comprueba para cada bloque marcado como extraído que sea de superficie o que sus 9 predecesores también estén extraídos.

6. Implementación preliminar

Se implementó en C++ una primera versión funcional de ambos algoritmos, compartiendo la misma estructura de datos, funciones de carga de instancias y construcción de precedencias, de modo que la comparación experimental no quede sesgada por diferencias de implementación.

Posteriormente, `common.hpp` se extendió para leer directamente el archivo entregado por el profesor. `loadInstance()` detecta automáticamente, a partir del número de columnas de la primera fila de datos, si la instancia utiliza el formato `x,y,z,v` descrito en el documento inicial del proyecto o el formato del archivo `escenario00` (`x,y,z,tonelada,metal_1,metal_2,roca`)

En el segundo caso, el valor económico de cada bloque se calcula internamente con la fórmula entregada por el profesor:

$$\text{ValorBloque} = (\text{Metal}_1 \times 0.89 \times 13228) - (\text{Tonelada} \times 13) - (\text{Tonelada} \times 25)$$

Utilizando constantes centralizadas en `common.hpp` correspondientes al costo de extracción, costo de procesamiento, recuperación y precio efectivo de cada metal. Esto permite ajustar los parámetros económicos sin modificar los algoritmos. Ambos algoritmos operan de la misma manera sobre el Model resultante, independientemente del formato de origen.

Los archivos entregados junto con el informe son:

- src/common.hpp — estructura Block/Model, carga de instancias CSV, detección automática de formato y construcción de precedencias.
- src/baseline.cpp — línea base basada en fuerza bruta.
- src/improved.cpp — algoritmo mejorado Floating Cone Incremental.
- src/generate_instance.cpp — generador de instancias sintéticas con semilla fija.
- src/validate.cpp — validador independiente de precedencia.
- data/ — instancias I1 a I4, casos de prueba manuales y escenario00_real.csv.
- results/ — resultados crudos de los experimentos (benchmark.csv, escalabilidad.csv y escalabilidad.png).
- README.txt — instrucciones detalladas de compilación y ejecución.

7. Resultados preliminares

7.1 Validación de correctitud sobre casos de prueba manuales

Siguiendo el protocolo definido en la Entrega 1, se construyeron cuatro casos pequeños con solución conocida de antemano.

Caso	Descripción	Resultado esperado	Baseline	Mejorado
1	Cono positivo simple (9 predecesores de superficie)	Vtotal = 43; 10 bloques extraídos	OK (43.000)	OK (43.000)
2	Bloque no extraíble (falta 1 de 9 predecesores)	Objetivo no extraído; 8 conos de superficie	OK (10.000; 1 inextraíble)	OK (10.000; 1 inextraíble)
3	Cono de valor exactamente cero	Cono no extraído; se reporta en conosCero	OK (18.000; 1 cono cero)	OK (18.000; 1 cono cero)
4	Recursión multinivel (3 niveles)	19 bloques del modelo; valor de prueba 16	OK (16.000)	OK (16.000)

Ambos algoritmos coinciden exactamente en los cuatro casos y fueron verificados con el validador independiente de precedencia, sin detectar violaciones.

7.2 Comparación baseline vs. mejorado en instancias sintéticas

Se generaron cuatro instancias sintéticas con semilla fija, simulando un yacimiento con mayor probabilidad de mineral cerca de la superficie y de estéril en profundidad. Cada configuración se ejecutó cinco veces y se reportan el promedio y la desviación estándar del tiempo de ejecución.

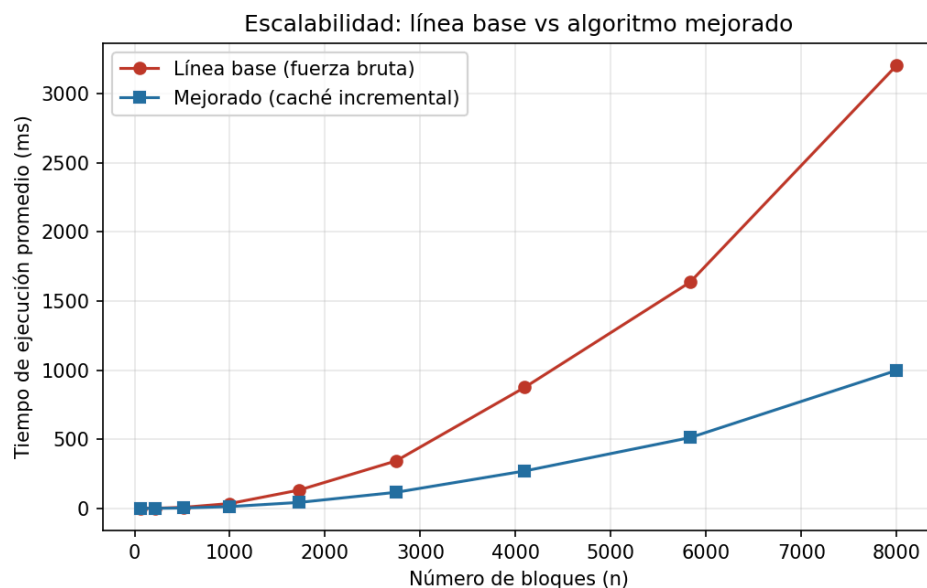
Instancia	n	Algoritmo	t promedio (ms)	σ (ms)	Vtotal	Bloques extr.	Conos
I1 (3×3×2)	18	Baseline	0.004	0.000	60.948	10	1
I1 (3×3×2)	18	Mejorado	0.011	0.001	60.948	10	1
I2 (5×5×5)	125	Baseline	0.137	0.009	133.346	35	1
I2 (5×5×5)	125	Mejorado	0.125	0.003	133.346	35	1
I3 (10×10×10)	1000	Baseline	35.884	0.641	1276.125	220	4
I3 (10×10×10)	1000	Mejorado	13.474	0.302	1325.298	212	4
I4 (15×15×15)	3375	Baseline	240.135	1.592	3661.345	680	1
I4 (15×15×15)	3375	Mejorado	164.143	1.507	3661.345	680	1

Las observaciones principales son las siguientes:

- En instancias pequeñas (I1 e I2), el algoritmo mejorado no tiene una ventaja clara a la línea base. En I1 incluso es marginalmente más lento, porque el costo fijo de mantener la caché y el índice inverso domina el ahorro de recomputación.
- A partir de I3, con 1000 bloques, el algoritmo mejorado muestra una mejora clara: aproximadamente 2.7 veces más rápido que la línea base. En I4 la mejora es de aproximadamente 1.5×.
- En I3, la política de mayor valor obtiene $V_{total} = 1325.298$ frente a 1276.125 de la línea base, extrayendo 8 bloques menos. Esto es consistente con la influencia de la política de selección sobre el resultado final.
- En I1, I2 e I4 ambas políticas coinciden en el valor final.

7.3 Escalabilidad

Para observar la tendencia de crecimiento con más puntos que los definidos en el protocolo original, se generaron instancias cúbicas adicionales de lado 4 a 20, equivalentes a 64 a 8000 bloques, ejecutando cada una tres veces con la misma semilla.



La línea base exhibe un crecimiento superlineal, consistente con la cota $O(n^3)$ del análisis teórico. Entre 1000 y 8000 bloques, el tamaño aumenta 8× y el tiempo aumenta cerca de 91×. El algoritmo mejorado también crece de forma superlineal, dado que su peor caso teórico no cambió, pero presenta una pendiente considerablemente menor debido a que evita recomputar candidatos no afectados en cada iteración. La brecha entre ambas curvas se amplía a medida que crece el modelo.

Por restricciones de tiempo, este barrido utilizó tres repeticiones en lugar de las cinco definidas en el protocolo de la Entrega 1 y no se midió memoria.

7.4 Uso de memoria

La medición de memoria máxima mediante `/usr/bin/time` confirma el trade-off anticipado en la sección 4.2. En I4, el algoritmo mejorado utiliza aproximadamente 5.1 MB frente a 4.6 MB de la línea base, es decir, alrededor de un 13% más, debido a la caché de conos y al índice inverso. En instancias pequeñas la diferencia es despreciable. Este costo adicional se considera aceptable para el tamaño de las instancias esperadas, aunque deberá revisarse si la instancia real es sustancialmente mayor.

7.5 Validación sobre el escenario real entregado por el profesor

Con el soporte de formato descrito en la sección 6, ambos algoritmos se ejecutaron directamente sobre `escenario00.csv`, el escenario real de 14.040 bloques entregado por el profesor. El valor económico de cada bloque se calculó a partir de tonelaje y leyes de metal según la fórmula provista.

Algoritmo	n	Vtotal	Bloques extr.	Conos	Inextraíbles	Tiempo (ms)
Baseline	14040	184 723 912.680	3 830	134	1 560	15 494.609
Mejorado	14040	206 641 384.121	3 275	20	1 560	2 816.061

Ambos algoritmos coincidieron en el número de bloques inextraíbles (1.560) y no detectaron conos de valor exactamente cero. El validador independiente no encontró violaciones de precedencia.

Sobre esta instancia real, considerablemente más grande e irregular que I4, la ventaja de tiempo del algoritmo mejorado fue aproximadamente de 5.5 con respecto a la línea base: 15.494,609 ms frente a 2.816,061 ms. Además, la política de mayor valor produjo un Vtotal sustancialmente mayor (206,6 M frente a 184,7 M), aunque extrajo menos bloques (3.275 frente a 3.830). Esto refuerza la relevancia de la política de selección en el resultado final.

8. Dificultades identificadas

- Complejidad de implementación: el índice inverso y el manejo de versiones para el borrado perezoso del heap fueron más difíciles de depurar que la línea base. Un primer intento sin versionado producía condiciones lógicas de extracción de conos parcialmente invalidados.
- Costo de memoria: la mejora de tiempo tiene un costo de memoria que crece con el grado de solapamiento entre conos. En modelos con mucho solapamiento podría ser necesario liberar cachés de candidatos con baja probabilidad de ser extraídos pronto.
- Diferencias de resultado entre políticas de selección: el hecho de que baseline y mejorado entreguen valores distintos en I3 demuestra que la política de selección influye de manera importante en el resultado.
- Generación de instancias representativas: los datos sintéticos son una aproximación razonable pero simplificada de un yacimiento real. Los resultados deben revalidarse considerando la distribución de valores y estructuras del escenario entregado por el profesor.

9. Plan de refinamiento para la Entrega 3

Para la siguiente entrega se establece el plan de trabajo, orientado a completar la evaluación del algoritmo, fortalecer su validación y consolidar la versión final del proyecto.

Las actividades consideradas son las siguientes:

- Analizar por separado el impacto de la caché y de la política de selección, para identificar cuánto aporta cada una a la mejora obtenida y comprender mejor su influencia en el rendimiento.
- Comparar distintas estrategias de selección de conos, evaluando cuál ofrece el mejor equilibrio entre tiempo de ejecución y valor económico obtenido.
- Evaluar una estrategia de liberación de caché para acotar el uso de memoria en instancias grandes, y medir su impacto en tiempo de ejecución.
- Actualizar el manejo de las instancias de entrada, ya que durante el desarrollo se implementó compatibilidad con dos formatos de archivos CSV. Inicialmente se trabajó con la estructura (x,y,z,v) indicada en el documento del proyecto; posteriormente, el profesor definió mediante un correo oficial el formato (x,y,z,Tonelada,Metal 1, Metal 2, Roca). Para la entrega final se revisará el código con el fin de mantener únicamente el formato definitivo y simplificar su implementación.

10. Conclusión

En esta entrega se diseñó e implementó el algoritmo Floating Cone Incremental como mejora de la línea base definida previamente. La propuesta conserva los conos calculados, utiliza un índice inverso para identificar los candidatos afectados y actualiza de forma eficiente la cola de prioridad después de cada extracción. Su complejidad temporal de peor caso continúa siendo $O(n^3)$ y su complejidad espacial puede alcanzar $O(n^2)$, pero evita recalcular todos los conos cuando las extracciones afectan solo a una parte de ellos.

Las pruebas realizadas mostraron que el algoritmo funciona correctamente en los distintos escenarios considerados, incluyendo conos positivos, bloques no extraíbles, conos de valor cero y casos con dependencias multinivel. Además, el rendimiento mejora a medida que aumenta el tamaño de las instancias: en las pruebas sintéticas se alcanzó una aceleración de hasta $3,21\times$ para 8000 bloques y, en el escenario real de 14040 bloques, una aceleración aproximada de $5,50\times$. Como contraparte, estas mejoras implican un mayor consumo de memoria debido al almacenamiento de información adicional para acelerar los cálculos.

Los resultados también mostraron que la estrategia utilizada para seleccionar los conos influye en la calidad de la solución obtenida. En las instancias I3 y en el escenario real, seleccionar el cono de mayor valor permitió obtener un mayor beneficio económico extrayendo menos bloques que la estrategia del primer cono positivo. Sin embargo, estos resultados corresponden únicamente a los casos evaluados y no permiten afirmar que una estrategia sea siempre superior. Por ello, en la siguiente entrega se ampliará la evaluación experimental y se compararán nuevas variantes antes de establecer una conclusión definitiva.

11. Bibliografía

Ares, G., Castañón Fernández, C., Diego Álvarez, I., Arias, D., & Bolaño Benítez, A. (2022). Open pit optimization using the floating cone method: A new algorithm. *Minerals*, 12(4), 495. <https://doi.org/10.3390/min12040495>

Dowd, P. A., & Onur, A. H. (1993). Open-pit optimization – Part 1: Optimal open-pit design. *Transactions of the Institution of Mining and Metallurgy, Section A: Mining Industry*, 102, A95–A104.

Hochbaum, D. S. (2008). The pseudoflow algorithm: A new algorithm for the maximum-flow problem. *Operations Research*, 56(4), 992–1009. <https://doi.org/10.1287/opre.1080.0524>

Lerchs, H., & Grossmann, I. F. (1965). Optimum design of open-pit mines. *Transactions of the Canadian Institute of Mining and Metallurgy*, 68, 17–24.

Picard, J.-C. (1976). Maximal closure of a graph and applications to combinatorial problems. *Management Science*, 22(11), 1268–1272. <https://doi.org/10.1287/mnsc.22.11.1268>

Turan, G., & Onur, A. H. (2023). Optimization of open-pit mine design and production planning with an improved floating cone algorithm. *Optimization and Engineering*, 24, 1157–1181. <https://doi.org/10.1007/s11081-022-09725-4>